

Instructions: Parts of the question answers should be together.

Answer all subparts (a–e) of Question 1 must be together. Each response should include the sorting method name and a brief explanation within 2–3 lines.

1. a. A warehouse has packages with weights ranging from **1 to 1,000,000**, but only **1000 packages**. Is **Counting Sort** a good choice? Why or why not? 1.5*5
 - b. A lab needs to sort DNA strings based on the length of string. Assume the strings are very long (length up to 10^6). Which **sorting** algorithm is most efficient?
 - c. A warehouse partitions packages in such a way that all lighter than pivot $\rightarrow$ left and all heavier $\rightarrow$ right. Many packages have **same weight as pivot**. How does this affect performance? How can you improve Quick Sort?
 - d. An embedded system needs sorting which is having very limited memory and cannot allocate extra arrays. For such a system, why is **Quick Sort** preferred over **Merge Sort**?
 - e. A company sorts employee records according to their monthly salary received. As many employees credited the same salary amount at end of the month. Sort the records in such a way that same priority items (same salary) must preserves the relative order of same salary. Is **Heap** sort being a good choice? Why or why not.
2. Arrange the following functions in the increasing order of their rate of growth: 1.5
 i) $(\log \log n)^{\log n}$ ii) $(\log n)^{1/2}$ iii) $\log(n)^{1/2}$ iv) $(\log n)^{\log n}$ v) $n^{\log \log n}$
3. Find the time complexity for the given iterative pseudocodes. 2*2

<pre> A(n) { for (i=2; i<=n; i=i*i) Printf("Hello") }</pre>	<pre> A(n) { for (i=1; i<=n; i++) { for (j=1; j<=n; j=j+i) { printf("Hello") } } }</pre>
--	--
 4. A recursive algorithm is defined as follows: 3
 An algorithm processes an input of size n . It makes **two recursive calls**, each on an input of size $\sqrt{n}$, and performs **constant work** outside the recursive calls.
 1. Write the recurrence relation for the running time $T(n)$.
 2. Find the **tight asymptotic bound** of $T(n)$.
 3. A student claims that:

“Since the height of the recursion tree is $O(\log \log n)$, so the total time complexity will be $O(\log \log n)$,” Is the student correct? Justify your answer.

- 5 a. Given an array of n elements. Write an algorithm to find three elements in the array such that their **sum** is equal to **k**.? Also discuss the time complexity. 2.5*2
- b. Given an array of elements, write an algorithm that segregates **even** and **odd** numbers. Algorithm should put all the **even** numbers first, then **odd** numbers.
(Hint: Input = < 12, 34, 45, 9, 8, 90, 3> output = <12, 34, 90, 8, 9, 45, 3>)
6. In the Maurya Empire, the royal treasury stores gold coins of different values in a long row. The King wants the coins arranged in **ascending order of value** before a grand ceremony. However, the royal minister suggests using a special method called the “**Royal Heap Method**”, where: 4
- The coins are first arranged into a **max-heap structure** (largest value at the top).
 - Then, the largest coin is repeatedly removed and placed at the end of the row.
 - The remaining coins are reorganized each time to maintain the heap property.

The initial coins are arranged as follows: {8, 3, 6, 5, 1, 2, 4}

Answer the following:

1. Perform the max **heapify** operation step by step for each recursive call and show the intermediate operations.
 2. Show the **extract max** element in Sorting process
 3. Analyse the time complexity and space complexity of this variant of Heap Sort.
7. Consider the **Quick Sort** algorithm where the pivot is always chosen as the **last element** (as discussed in class). The partition divides the array such that elements smaller than the pivot are placed to its left and greater elements to its right. 5

Given the array: $A = \{8, 3, 6, 5, 1, 2, 4\}$

Answer the following:

1. Perform the **PARTITION** operation step-by-step for each recursive call and show the intermediate arrays.
2. Draw the **recursion tree** for the execution of Quick Sort on the given array.
3. Analyse the time complexity and space complexity of this variant of Quick Sort.
4. Determine the height of the recursion tree in terms of n for best and worst cases.
5. Explain why **Merge Sort may perform slower than Quick Sort in practice for small input sizes**, despite having the same asymptotic complexity.

CS409: Design and Analysis of Algorithms

Maximum Time: 3 Hours

Sem: II

Maximum Marks: 50

All questions are compulsory. Parts of the question answers should be together.

- 1 Consider the following asymptotic functions (3)

$$\text{Let } f(n) = \begin{cases} n^4 & 0 < n < 2,000 \\ n^{\log(8)} & n \geq 2,000 \end{cases} \quad \text{and} \quad g(n) = \begin{cases} n^5 \log n & 0 < n < 1000 \\ n^{\sqrt[4]{4}} & n \geq 1000 \end{cases}$$

Is $f(n)$ is $O(g(n))$. If yes then find the value of constants C and n_0 otherwise justify.

- 2 Is the following recurrence for the knapsack problem correct? If not, give the correct recurrence. Justify your answer in either case. (3)

If $w < w_i$, then $\text{OPT}(i, w) = \text{opt}(i-1, w)$

Otherwise $\text{OPT}(i, w) = \max(\text{opt}(i-1, w), v_i + \text{opt}(i-1, w))$

- 3 Consider an instance of weightage interval scheduling with 6 intervals as specified below: (4)

Interval Number	Starting time	Finishing time	Weight
1	0	2	2
2	1	3	4
3	2	4	4
4	1	5	7
5	4	5	2
6	4	6	1

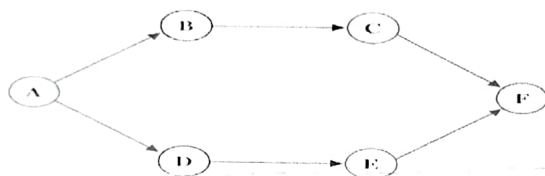
With the help of the above example, argue that the memoized recursive algorithm solves a lesser number of subproblems than the corresponding iterative algorithm.

- 4 Consider the rabbit reproduction problem based on Fibonacci Model: Initially, a single pair (unmature) of rabbits is placed in an isolated area. Each pair of rabbits becomes mature after one month and, from the second month onward, produces a new pair every month. No rabbits ever die. (3)
- a. Define the recurrence relation for the number of rabbit pairs after n months. (3)
- Also compute the number of pairs (mature and unmature) after 6 months.
- b. Explain why this method is more efficient than Dynamic Programming for large n . (2)
- 5 A student claims that the minimum and maximum of an array can be found in $(n-1)$ comparisons. (2)
- a. Is the statement correct? Justify your answer. (2)
- b. What is the minimum number of comparisons required to find both the minimum and the maximum in terms of n ? How many comparisons required to find minimum and maximum for $n=1000$. (3)

- 6 A university has a single seminar hall. There are n booking requests, each with a start time (s_i) and finish time (f_i). Where i represents the i^{th} activity.

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

- Formulate this problem as an Activity Selection Problem. Apply a greedy strategy to select the maximum number of non-overlapping seminars. (3)
 - The activities are selected based on the shortest duration (finish – start) instead of the earliest finish time. Is this strategy correct? Justify your answer. (2)
- 7 Consider the sum of subsets problem on given an array $A < 2, 3, 7, 8, 10, 12 >$
Answer the following:
- Find the number of subsets whose sum is equal to 12 using Dynamic programming. (2)
 - How many subsets exist such that the difference between two subsets sums equal to 2? Both subsets contain all the elements. (3)
- 8 Consider a directed graph $G(V, E)$



- How many topological orderings does this graph have? If multiple valid orders exist, write all and justify. (3)
 - Consider the given graph (treat all edges as undirected and of equal weight). How many distinct minimum spanning trees (MSTs) exist? (2)
- 9 Consider the given Text (**T**): abcabcabcabcabc and Pattern (**P**): abcabc for string matching.
- Construct the LPS (Longest Prefix Suffix) array for the given pattern abcabc. (2)
 - Apply the KMP Algorithm using the LPS array on the given text **T**. Trace the **P**. (3)
- 10 Construct a Bitonic sequence from the following input:
3, 7, 4, 8, 6, 2, 1, 5
- Show all the steps for splitting it into increasing and decreasing parts. (3)
 - What is the time complexity of Bitonic Sort? (2)
- 11 Solve the 4-Queen problem using the backtracking method and show all possible solutions. Also draw the state space tree. (5)

Instructions: Parts of the question answers should be together.

Answer all subparts (a–e) of Question 1 must be together. Each response should include the sorting method name and a brief explanation within 2–3 lines.

1. a. A warehouse has packages with weights ranging from **1 to 1,000,000**, but only **1000 packages**. Is **Counting Sort** a good choice? Why or why not? 1.5*5

No counting sort is not good choice because the range is greater than the number of packages. So time complexity reaches to the $O(n^2)$

- b. A lab needs to sort DNA strings based on the length of string. Assume the strings are very long (length up to 10^6). Which **sorting** algorithm is most efficient?

Radix sort will be a good choice here because the sequence is very large. this works exceptionally well for large datasets with integer keys (lengths).

- c. A warehouse partitions packages in such a way that all lighter than pivot $\rightarrow$ left and all heavier $\rightarrow$ right. Many packages have **same weight as pivot**. How does this affect performance? How can you improve Quick Sort?

The many values are same as pivot so it effects the performance to give the time complexity near to $O(n^2)$. So randomized quicksort will be a better choice here.

- d. An embedded system needs sorting which is having very limited memory and cannot allocate extra arrays. For such a system, why is **Quick Sort** preferred over **Merge Sort**?

Better **cache efficiency, in-place sorting and** requires space for the recursion stack rather than the extra space used by merge sort.

- e. A company sorts employee records according to their monthly salary received. As many employees credited the same salary amount at end of the month. Sort the records in such a way that same priority items (same salary) must preserves the relative order of same salary. Is **Heap** sort being a good choice? Why or why not.

Heap sort is not stable sorting so it not gives the guarantees to preserve the relative order.

2. Arrange the following functions in the increasing order of their rate of growth: 1.5

i) $(\log \log n)^{\log n}$ ii) $(\log n)^{1/2}$ iii) $\log(n)^{1/2}$ iv) $(\log n)^{\log n}$ v) $n^{\log \log n}$

$(\log n)^{1/2} = \log(n)^{1/2} < (\log \log n)^{\log n} < (\log n)^{\log n} = n^{\log \log n}$

ii = iii < I < iv = v

3. Find the time complexity for the given iterative pseudocodes. 2*2
- | | |
|--|---|
| <pre> A(n) { for (i=2; i<=n; i=i*i) Printf("Hello") } </pre> <p>1st: $i = 2$
 2nd: $i = 2^2 = 4$
 3rd: $i = 4^2 = 16 = 2^{2^2}$
 4th: $i = 16^2 = 256 = 2^{2^3}$</p> <p>After k^{th} iteration $n=2^{2^k}$ so $k = \log \log n$ and it will execute $k+1$ times so time complexity will be $O(\log \log n)$</p> | <pre> A(n) { for (i=1; i<=n; i++) { for (j=1; j<=n; j=j+i) { printf("Hello") } } } </pre> <p>When $i=1$: inner loop runs n times
 When $i=2$: inner loop runs $n/2$ times
 When $i=3$: inner loop runs $n/3$ times
 When $i=k$: inner loop runs n/k times
 When $i=n$: inner loop runs 1 times
 So total behave like
 $n(1+1/2+1/3+1/4+\dots+1/n)$
 time complexity will be $O(n \log n)$</p> |
|--|---|
4. A recursive algorithm is defined as follows: 3
- An algorithm processes an input of size n . It makes **two recursive calls**, each on an input of size $\sqrt{n}$, and performs **constant work** outside the recursive calls.
1. Write the recurrence relation for the running time $T(n)$. $T(n)=2T(\sqrt{n})+1$
 2. Find the **tight asymptotic bound** of $T(n)$.
 $T(n)=T(n^{1/2})+1$, $T(n^{1/2})=T(n^{1/4})+1$ and $T(n^{1/4})=T(n^{1/8})+1$ after 3rd iteration
 $T(n)=2^3T(n^{1/2^3})+4+2+1$ so after k^{th} iteration $T(n)=2^kT(n^{1/2^k})+2^{k-1}+2^{k-2}+\dots+2^2+2^1+2^0$
After solving the above $T(n)=O(\log n)$
 3. A student claims that:
"Since the height of the recursion tree is **$O(\log \log n)$** , so the total time complexity will be **$O(\log \log n)$** ," Is the student correct? Justify your answer.
No, see the point no 2
- 5 a. Given an array of n elements. Write an algorithm to find three elements in the array such that their **sum** is equal to **k**.? Also discuss the time complexity. 2.5*2
- b. **BruteForceSearch(A, n, k)**
For(a=0; a<n; a++)
 For(b=0; b<n; b++)
 For(c=0; c<n; c++)
 If(A[a]+A[b]+A[c]==k)
 Print the values of a, b, c; $T(n)=n^3$

Given an array of elements, write an algorithm that segregates **even** and **odd** numbers. Algorithm should put all the **even** numbers first, then **odd** numbers.

(Hint: Input = < 12, 34, 45, 9, 8, 90, 3> output = <12, 34, 90, 8, 9, 45, 3>)

1. Initialize two index variables left and right: left=0 and right= n-1
2. Keep incrementing the left index until see you an odd number.
3. Keep decrementing the right index until see you an even number.
4. If left < right then swap (A[left] and A[right])

6. In the Maurya Empire, the royal treasury stores gold coins of different values in a long row. The King wants the coins arranged in **ascending order of value** before a grand ceremony. However, the royal minister suggests using a special method called the “**Royal Heap Method**”, where: 4

- The coins are first arranged into a **max-heap structure** (largest value at the top).
- Then, the largest coin is repeatedly removed and placed at the end of the row.
- The remaining coins are reorganized each time to maintain the heap property.

The initial coins are arranged as follows: **<8, 3, 6, 5, 1, 2, 4>**

Answer the following:

1. Perform the max **heapify** operation step by step for each recursive call and show the intermediate operations.
T1:<8, 3, 6, 5, 1, 2, 4>; T2: <8, 3, 6, 5, 1, 2, 4>; T3:<8, 5, 6, 3, 1, 2, 4> T4:<8, 5, 6, 3, 1, 2, 4>
2. Show the **extract max** element in Sorting process
Extract the max from from tree 4 i.e. T4: Extract 8 from three and replace with last node 4 and apply the heapify. Tree will be <6, 5, 4, 3, 1, 2> and so on....
3. Analyse the time complexity and space complexity of this variant of Heap Sort.
Time complexity $O(n \log n)$ space complexity $O(1)$

7. Consider the **Quick Sort** algorithm where the pivot is always chosen as the **last element** (as discussed in class). The partition divides the array such that elements smaller than the pivot are placed to its left and greater elements to its right. 5

Given the array: **A = <8, 3, 6, 5, 1, 2, 4>**

Answer the following:

1. Perform the **PARTITION** operation step-by-step for each recursive call and show the intermediate arrays.
After first partition the position of elements are <3, 1, 2, 4, 8, 6, 5>
After second partition the position of elements are <1, 2, 3, 4, 8, 6, 5>
After second partition the position of elements are <1, 2, 3, 4, 5, 6, 8>

2. Draw the **recursion tree** for the execution of Quick Sort on the given array.
Draw the recursion tree according to the above partitions
3. Analyse the time complexity and space complexity of this variant of Quick Sort. Best case: $T(n) = (n \log n)$ Worst case: $T(n) = (n^2)$
4. Determine the height of the recursion tree in terms of n for best and worst cases.
5. Explain why **Merge Sort may perform slower than Quick Sort in practice for small input sizes**, despite having the same asymptotic complexity.

Better **cache efficiency, in-place sorting and** requires space for the recursion stack rather than the extra space used by merge sort.

CS409: Design and Analysis of Algorithms

Maximum Time: 3 Hours

Sem: II

Maximum Marks: 50

All questions are compulsory. Parts of the question answers should be together.

- 1 Consider the following asymptotic functions (3)

$$\text{Let } f(n) = \begin{cases} n^4 & 0 < n < 2,000 \\ n^{\log(8)} & n \geq 2,000 \end{cases} \quad \text{and} \quad g(n) = \begin{cases} n^5 \log n & 0 < n < 1000 \\ n^{\sqrt{4}} & n \geq 1000 \end{cases}$$

Is $f(n)$ is $O(g(n))$. If yes then find the value of constants C and n_0 otherwise justify.

No,

Because $f(n)=n^3$ when $n_0=2000$ and $g(n)=n^2$ so $f(n)$ is not bigoh of $g(n)$.

- 2 Is the following recurrence for the knapsack problem correct? If not, give the correct recurrence. Justify your answer in either case. (3)

If $w < w_i$, then $\text{OPT}(i, w) = \text{opt}(i-1, w)$

Otherwise $\text{OPT}(i, w) = \max(\text{opt}(i-1, w), v_i + \text{opt}(i-1, w))$

No, pls justify in 2-3 lines

Correct is If $w < w_i$, then $\text{OPT}(i, w) = \text{opt}(i-1, w)$

Otherwise $\text{OPT}(i, w) = \max(\text{opt}(i-1, w), v_i + \text{opt}(i-1, w-w_i))$

- 3 Consider an instance of weightage interval scheduling with 6 intervals as specified below: (4)

Interval Number	Starting time	Finishing time	Weight
1	0	2	2
2	1	3	4
3	2	4	4
4	1	5	7
5	4	5	2
6	4	6	1

With the help of the above example, argue that the memoized recursive algorithm solves a lesser number of subproblems than the corresponding iterative algorithm.

See the class notes $\text{opt}(j)=0,0,1,0,3,3$ then find $\text{OPT}(j)=\max(w_j+\text{opt}(p(j)), \text{opt}(j-1))$ so time complexity for recursive tree is 2^n . so $M_{\text{compute_OPT}}(j)$ i.e.

$M[j]=\max(w_j+\text{compute_OPT}(p(j)), \text{compute_OPT}(j-1))$ see the class example

- 4 Consider the rabbit reproduction problem based on Fibonacci Model: Initially, a single pair (unmature) of rabbits is placed in an isolated area. Each pair of rabbits becomes mature after one month and, from the second month onward, produces a new pair every month. No rabbits ever die.

- a. Define the recurrence relation for the number of rabbit pairs after n months. (3)
Also compute the number of pairs (mature and unmatue) after 6 months.
 $f(n)=f(n-1)+f(n-2)$ $n>2$ with base condition $f(1)=1$ and $f(2)=2$

Month	Mature	Unmature	Total pairs
1	0	1	1
2	1	0	1
3	1	1	2
4	2	1	3
5	3	2	5
6	5	3	8

- b. Explain why this method is more efficient than Dynamic Programming for large n . (2)
The Fibonacci relation can be represented using matrices:

$$\begin{bmatrix} F(n) \\ F(n-1) \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 0 \end{bmatrix}^{n-1} \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Instead of computing all previous Fibonacci numbers one by one, matrix exponentiation uses **fast exponentiation** (divide-and-conquer).

This drastically reduces the number of multiplications.

- 5 A student claims that the minimum and maximum of an array can be found in $(n-1)$ comparisons. (2)
a. Is the statement correct? Justify your answer. **No pls justify** (2)
b. What is the minimum number of comparisons required to find both the minimum and the maximum in terms of n ? How many comparisons are required to find the minimum and maximum for $n=1000$? (3)

for n is even No of comparisons $= (3n/2) - 2$; when n is odd No of comparisons $= (3n/2) - 3/2$ for $n=1000$ the no of comp $= 1498$

- 6 A university has a single seminar hall. There are n booking requests, each with a start time (s_i) and finish time (f_i). Where i represents the i^{th} activity.

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

- a. Formulate this problem as an Activity Selection Problem. Apply a greedy strategy to select the maximum number of non-overlapping seminars. (3)
b. The activities are selected based on the shortest duration (finish – start) instead of the earliest finish time. Is this strategy correct? Justify your answer. (2)

Sort Activities by Earliest Finish Time, solve as did in class, Optimal activities using standard greedy method: $\{1, 4, 8, 11\}$

Selecting Activities Based on Shortest Duration

Now duration is:

$$d_i = f_i - s_i$$

Is Shortest Duration Strategy Always Correct?

No, it is not always correct

give counter example to justify.

7 Consider the sum of subsets problem on given an array $A = \langle 2, 3, 7, 8, 10, 12 \rangle$

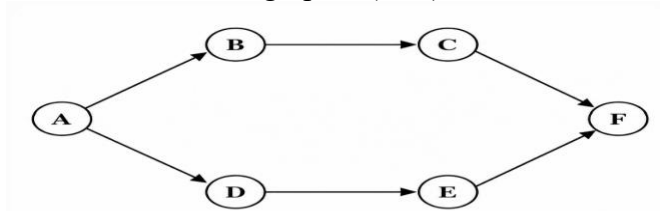
Answer the following:

a. Find the number of subsets whose sum is equal to 12 using Dynamic programming. (2)

b. How many subsets exist such that the difference between two subsets sums equal to 2? Both subsets contain all the elements. (3)

See the class notes

8 Consider a directed graph $G(V, E)$



a. How many topological orderings does this graph have? If multiple valid orders exist, write all and justify. (3)

A,B,C,D,E,F

A,B,D,C,E,F

A,B,D,E,C,F

A,D,E,B,C,F

A,D,B,E,C,F

A,D,B,E,E,F

b. Consider the given graph (treat all edges as undirected and of equal weight). How many distinct minimum spanning trees (MSTs) exist? (2)

NO OF MINIMUM SPANNING TREES=6

9 Consider the given Text (**T**): abcabcabcabcabc and Pattern (**P**): abcabc for string matching.

a. Construct the LPS (Longest Prefix Suffix) array for the given pattern abcabc. (2)

b. Apply the KMP Algorithm using the LPS array on the given text **T**. Trace the **P**. (3)

See the class notes

10 Construct a Bitonic sequence from the following input:

3, 7, 4, 8, 6, 2, 1, 5

a. Show all the steps for splitting it into increasing and decreasing parts. (3)

b. What is the time complexity of Bitonic Sort? (2)

See the class notes

11 Solve the 4-Queen problem using the backtracking method and show all possible solutions. Also draw the state space tree. (5)

See the class notes